

PROJECT PROPOSAL

AI-Powered Applicant Tracking System

A semantic, explainable recruiting platform — turning resumes into structured knowledge and ranking the best candidates with RAG + LLM reasoning.



Semantic candidate search — not keyword matching

Explainable rankings with evidence for every decision

Hidden-gem detection that defeats years-of-experience bias

1

Executive Summary

We propose building an **AI-Powered Applicant Tracking System (ATS)** that converts raw resumes into structured candidate profiles, indexes them with semantic embeddings, and ranks applicants against recruiter queries or job descriptions using retrieval-augmented generation (RAG) plus LLM reasoning.

The system replaces today's **keyword-matching shortlisting** with **semantic, explainable, bias-aware ranking**. Recruiters get not only a ranked list — they get *why* each candidate matches, backed by citations to the resume. High-potential candidates with fewer years of experience are no longer discarded.

Why now

- Manual screening consumes ~40% of recruiter time before first contact.
- Keyword filters reject strong candidates who phrase experience differently.
- LLM + vector-database tech is mature, cheap, and explainable enough for production.

What we deliver

- A production-ready, deployable application in ~15 weeks (5 phases).
- Resume ingestion, semantic search, explainable ranking, and a modern recruiter UI.
- Provider-agnostic design — swap AI models and infrastructure without code changes.

Core promise: every candidate rank is a *scored, explained, evidence-backed decision* — replacing "gut feel and grep" with auditable AI.

2

The Problem

Recruiting teams struggle with three compounding issues that slow hiring and lower quality:

Pain point	Impact
Keyword matching	Applicants are filtered by exact terms. A senior engineer who "built payments platforms" is invisible to a search for "banking experience". Strong candidates are lost before a human ever reads them.
No explainability	Recruiters can't tell <i>why</i> one candidate ranked above another. Rankings are a black box, hard to defend, audit, or improve.
Years-of-experience bias	Filters set minimum tenure, systematically excluding high-potential juniors and career-changers who bring fresh skills — even when they are the best fit.
Manual triage cost	Reading and comparing resumes by hand consumes 30–40% of recruiter time per role, directly inflating cost-per-hire and time-to-fill.

THE OPPORTUNITY

A semantic, explainable system removes the trade-off: it **searches meaning**, not words; **justifies every ranking**; and **actively surfaces hidden gems**. The result is faster shortlists, higher quality, fairer screening, and a defensible audit trail for every hiring decision.

3

The Proposed Solution

An end-to-end platform with two actors and one shared knowledge core.

CORE CAPABILITIES

Capability	Description
Smart resume ingestion	Upload PDF → extract text (with OCR for scanned files) → LLM structures it into strict JSON → skills are normalized to a canonical taxonomy → content is chunked and embedded into a vector index.
Natural-language search	Recruiters search the way they think: "senior Flutter engineer with banking payments experience". Semantic retrieval finds meaning, not exact keywords.
Explainable ranking	A deterministic scoring engine (skills, experience, education, certification, hidden-gem signals) sets the rank; an LLM explains it with strengths, weaknesses, a recommendation, and citations to evidence chunks.
Hidden-gem detection	An explicit potential model rewards skill depth, impactful projects, fast progression, leadership, and open-source work — so strong juniors are surfaced instead of filtered out.
Job-description matching	Paste a job description and get the best candidates for that exact role, ranked and explained.
Transparency & audit	Every ranking is persisted with scores, model versions, and evidence chunk IDs. All AI claims are traceable — a built-in hallucination guard.

HOW IT WORKS — HIGH LEVEL

Applicant flow (index) 1. Resume uploaded (PDF) 2. Text extraction + OCR fallback 3. LLM structures profile to JSON 4. Skill canonicalization 5. Semantic chunking + embedding 6. Vectors stored (Qdrant), profile in PostgreSQL	Recruiter flow (rank) 1. Natural-language query or job description 2. Query embedded → vector search 3. Candidate profiles hydrated 4. Deterministic scores → bucket assignment 5. LLM explanation (evidence-constrained) 6. Ranked list: best / strong / hidden gem / alternative
---	---

4

Technology Stack

We select proven, swappable components. Every AI dependency sits behind a provider interface, so models and infrastructure can be changed via configuration — never code.

Layer	Choice	Why
Frontend	Flutter (Material 3)	One codebase for web, iOS, and Android; modern recruiter & applicant UIs.
Backend	Python FastAPI	Async, typed, clean architecture; ideal for AI pipelines and REST APIs.
Relational DB	PostgreSQL	Source of truth for candidates, resumes, jobs, applications, and audit-logged rankings.
Vector DB	Qdrant	High-performance semantic index; idempotent upserts and snapshotting.
Embeddings	BAAI bge-small-en-v1.5	384-dim, CPU-friendly, strong retrieval at low cost; swappable to larger models.
LLM	Provider-agnostic	OpenAI, Gemini, Ollama, vLLM, or any OpenAI-compatible endpoint — chosen by config.
OCR	Tesseract / PaddleOCR	Scanned-PDF fallback.
Async processing	Background workers	Resume pipelines and notifications off the request hot path.

KEY DESIGN PRINCIPLES

- **Clean architecture** — business logic is isolated and testable, independent of frameworks and AI providers.
- **Provider abstraction** — embedding, LLM, vector store, and OCR are ports with adapters; swapping costs one config change.
- **Hallucination guard** — the LLM ranks *only* from retrieved evidence; every claim carries a chunk ID.
- **Bias-aware by design** — hidden-gem scoring is a transparent, auditable code-level model, not LLM discretion.

FastAPI

Flutter

PostgreSQL

Qdrant

BGE embeddings

RAG

Clean Architecture

5

Project Scope

INCLUDED IN THE MVP (PHASES 0–2, ~8 WEEKS)

- Foundation: monorepo, CI, Docker Compose (Postgres + Qdrant), provider plumbing.
- Resume ingestion: upload → extraction → LLM structuring → embeddings → vector index, with status tracking.
- Search & deterministic ranking: natural-language search, intent parsing, skill/experience/education/certification scores, bucket assignment, hidden-gem detector.
- Flutter screens: applicant upload, recruiter search, ranked candidate list, candidate detail.
- Ranking persistence for audit and caching.

INCLUDED IN THE PRODUCTION RELEASE (PHASES 3–4, ~7 MORE WEEKS)

- LLM reasoning tier: strengths / weaknesses / recommendation / evidence with hallucination guard.
- Job-description search and candidate comparison UI with evidence drill-down.
- Auth & RBAC (JWT), rate limiting, upload validation and virus scan.
- Observability: structured logs, tracing, metrics, error tracking.
- Resilience: retries, circuit breakers, DLQ, GDPR erasure, re-index jobs.
- Deployment: Terraform + CI/CD pipeline, staging environment.

Not in scope for v1 (post-launch backlog): interview-question generator, candidate fit scores, bias analytics, salary prediction, multilingual resumes, and AI resume improvement suggestions.

6

Delivery Roadmap

We build in **vertically-sliced, demoable milestones**. Each phase is runnable end-to-end — stakeholders see working software every 2–3 weeks.

Phase	Timeline	Goal	Exit criteria
0 · Foundation	Weeks 1–2	Repo, CI, Docker stack, API skeleton, DI container, provider ports, Flutter skeleton.	docker compose up runs API + worker; providers selectable by env.
1 · Resume Ingestion	Weeks 3–5	Upload → structured JSON profile → semantic chunks → Qdrant vectors.	10 varied sample PDFs (incl. 2 scanned) all reach INDEXED.
2 · Search + Ranking	Weeks 6–8	Natural-language query → ranked buckets (deterministic tier).	5 canned queries produce reasonable buckets; scores reproducible; rankings persisted.
3 · Explainability	Weeks 9–11	LLM reasoning tier + evidence UI + search-by-JD.	Explanations cite only retrieved chunks; eval harness runs.
4 · Production	Weeks 12–15	Auth/RBAC, observability, resilience, GDPR, IaC, CI/CD, staging deploy.	Staging live via pipeline; load test 100 resumes + 50 concurrent searches passes.
5 · AI Quality (ongoing)	Week 16+	Bias detection, analytics, and the post-launch feature backlog.	Per-feature; each gated by the eval suite.

SUGGESTED TEAM (FULL BUILD)

Role	Focus
Backend engineer ×2	FastAPI, resume pipeline, provider adapters
AI/ML engineer ×1	RAG tuning, eval harness, embedding selection
Flutter engineer ×1	All client screens and evidence UX
DevOps (part-time) ×1	IaC, CI/CD, observability
QA / Data ×1	Golden resume set, ranking evals, acceptance

7

Risks & Mitigations

Risk	Impact if unmitigated	Mitigation
LLM structuring errors on messy resumes	Broken profiles, silent data loss	Strict JSON schema + validation, self-healing retries, manual-review queue, golden eval set.
RAG quality on niche queries	Missed candidates	Optional hybrid BM25 + reciprocal rank fusion; eval harness; embedding A/B tests.
Provider cost spikes	Unpredictable budget	Token budgets, caching, cheap structuring model, local BGE as default embedder.
Model / provider lock-in	Inflexibility, inflated costs	Ports + adapters; providers selected by config; swap gated by the same golden eval suite.
Regulatory compliance (GDPR)	Fines, reputational harm	Right-to-erasure API, audit logs, data-retention policy built in from Phase 1.
AI hallucination / wrong claims	Poor hiring decisions	Evidence-constrained prompts; chunk-ID subset validation; raw scores always visible.
Hidden-gem over-suggesting	Noisy shortlists	Tunable thresholds; explainable reasons; recruiter feedback loop.

Design-first insurance: the full design package (architecture, ERD, API spec, AI/RAG/ranking pipelines) is completed *before* build — cutting rework and giving the team a shared blueprint from day one.

8

Success Metrics

We will measure both technical quality and business value. Each metric is defined up front so "done" is unambiguous.

Metric	Target	What it proves
Ranking quality (nDCG@20)	≥ 0.80	Semantic + LLM ranks match human expert judgment.
Pipeline success rate	≥ 95% upload → INDEXED	Resume ingestion is reliable across formats.
Time to index	Median < 30 s	Applicants see results fast; workers scale.
Search latency	p95 < 2 s	Recruiter experience stays responsive at scale.
Time-to-shortlist	Reduced ≥ 40%	Real recruiter-hour savings vs manual review.
Hidden-gem yield	≥ 15% of shortlist → interview	Potential-based ranking adds value beyond years.
Hallucination guard	0 unverifiable claims	Every explanation claim maps to a chunk.
Determinism	Reproducible ranks	Same query + data → same result, auditable.

INVESTMENT SUMMARY (INDICATIVE)

Estimates are planning figures for budget discussion; final numbers depend on team mix and infra.

Item	Estimate	Notes
Design & build effort	~15 team-weeks (5 phases)	Full team as in §6; ~\$ — adjust for rate card
AI/API run costs	Variable, low	Local BGE default; LLM tokens budgeted & cached
Infrastructure	Small	Postgres + Qdrant + app hosting; auto-scaling
Data & QA	Included in build	Golden resume set + labeled eval pairs

9

Next Steps & Call to Action

We are ready to start. The design is complete; the first vertical slice (Phase 0–1) can begin immediately.

TO MOVE FORWARD, WE ASK FOR

- **Approval** of this proposal and the 15-week roadmap.
- **Budget sign-off** on the indicative investment in §8.
- **Team allocation** — confirm the roles in §6.
- **A pilot dataset** — 20–30 sample resumes (a mix of digital and scanned PDFs) to validate ingestion early.

IMMEDIATE MILESTONES AFTER GO-AHEAD

Week	Deliverable
Week 1	Repo + CI + Docker stack running; API skeleton with health checks; provider wiring.
Week 3	First vertical slice: one resume fully ingested end-to-end (upload → profile → vectors) with status tracking.
Week 5	10 resumes indexed; structured profiles verified in PostgreSQL; vectors searchable.
Week 8	Natural-language search returns ranked buckets in the Flutter UI (MVP demo).

Contact: Product & Engineering · *placeholder@company.com* · Next review: within two weeks of approval.